



Databases and its Applications

Relational Modelling

Pooja T S
Computer Applications

Databases and its Applications

Relational Modelling

SQL: WHERE Clause, Aggregate Functions

Pooja T S

Computer Applications



Databases and its Applications

Purpose of the WHERE Clause

- ▶ The WHERE clause is used to filter rows returned by SELECT, UPDATE, or DELETE.
- ▶ It tells PostgreSQL: “consider only those rows that satisfy this condition.”
- ▶ Without WHERE, the statement affects or returns **all** rows of the table.



Databases and its Applications

Basic WHERE Syntax

- ▶ General form:
- ▶ `SELECT column_list FROM table_name WHERE condition;`
- ▶ Condition is built using comparison operators, logical operators, or special tests (NULL, BETWEEN, IN, LIKE).



Databases and its Applications

Comparison Operators in PostgreSQL

- ▶ Most common comparison operators:
 - = equal to
 - <> or != not equal to
 - > greater than, < less than
 - >=, <= greater / less than or equal
- ▶ These return TRUE/FALSE/UNKNOWN for each row; only TRUE rows are selected.



Databases and its Applications

Example – Simple WHERE

- ▶ Command:
- ▶ `college_db=# SELECT sid, sname, cgpa
FROM student
WHERE cgpa > 8.5;`

- ▶ Output (sample):

sid	sname	cgpa
2	Kavya	9.10

(1 row)



Databases and its Applications

Logical Operators

- ▶ PostgreSQL supports logical operators to build complex conditions:
 - AND – all conditions must be TRUE
 - OR – at least one condition must be TRUE
 - NOT – negates a condition
- ▶ Use parentheses () to control evaluation order.



Databases and its Applications

Example – AND / OR

- ▶ Command:

```
SELECT sname, deptid, cgpa  
FROM student  
WHERE deptid = 1 AND cgpa >= 8.0;
```
- ▶ Meaning: pick students from department 1 **and** with good CGPA.
- ▶ Change AND to OR to broaden the result.



Databases and its Applications

Operator Precedence

- ▶ In PostgreSQL NOT is evaluated first, then AND, then OR.
- ▶ If in doubt, always use parentheses.
- ▶ Example: `WHERE deptid = 1 AND (cgpa > 8 OR city = 'Bangalore')`



Databases and its Applications

Filtering on Text = and ILIKE

- ▶ For exact text match use =:
`WHERE sname = 'Ravi'`
- ▶ PostgreSQL also offers case-insensitive match using ILIKE:
`WHERE sname ILIKE 'ravi'`
- ▶ Useful when user input may vary in case.



Databases and its Applications

Special Predicate – BETWEEN

- ▶ BETWEEN checks if a value lies in an inclusive range.
- ▶ Syntax: `WHERE column BETWEEN low AND high`
- ▶ Equivalent to: `column >= low AND column <= high`



Databases and its Applications

Example – BETWEEN

- ▶ Command:
- ▶ `SELECT sname, cgpa FROM student
WHERE cgpa BETWEEN 7.0 AND 9.0;`

- ▶ Output (sample):

sname	cgpa
Ravi	8.40
Anita	7.80

(2 rows)



Databases and its Applications

Special Predicate – IN

- ▶ IN checks membership in a list of values.
- ▶ Syntax: `WHERE column IN (val1, val2, ...);`
- ▶ Easier than writing many OR conditions.



Example – IN

- ▶ Command:
- ▶ `SELECT sname, deptid FROM student
WHERE deptid IN (1,3);`

- ▶ Output (sample):

sname	deptid
Ravi	1
Anita	3

(2 rows)



Databases and its Applications

Pattern Matching – LIKE

- ▶ LIKE allows basic pattern matching.
 - % – matches any sequence of characters
 - _ – matches exactly one character
- ▶ Example: `WHERE sname LIKE 'K%'` → names starting with K



Databases and its Applications

Example – LIKE

- ▶ Command:

```
SELECT sname FROM student  
WHERE sname LIKE '%a';
```
- ▶ Output (sample):

sname
Kavya

(1 row)
- ▶ Meaning: names ending with a.



Databases and its Applications

Working with NULL

- ▶ NULL means “unknown / missing value”.
- ▶ = or <> cannot be used to test NULL.
- ▶ Use: IS NULL or IS NOT NULL



Databases and its Applications

Example – IS NULL

- ▶ Command:
- ▶ `SELECT sid, sname FROM student
WHERE deptid IS NULL;`
- ▶ This picks students whose department is not yet assigned.
- ▶ Similarly: `WHERE deptid IS NOT NULL`



Databases and its Applications

WHERE with UPDATE and DELETE

- ▶ Same filter idea works with other statements:
- ▶ `UPDATE student SET cgpa = 9.0 WHERE sid = 1;`
- ▶ `DELETE FROM student WHERE cgpa < 5.0;`
- ▶ Without WHERE, all rows are updated or deleted.



Databases and its Applications

What Are Aggregate Functions?

- ▶ Aggregates perform a calculation on a set of rows and return a single value.
- ▶ Used for reporting, statistics, summaries.
- ▶ In PostgreSQL, the common ones are COUNT, SUM, AVG, MIN, MAX.



Databases and its Applications

COUNT() – Counting Rows

- ▶ COUNT(*) counts all rows.
- ▶ COUNT(column) counts only non-NULL values in that column.
- ▶ Very useful for checking inserted data.



Databases and its Applications

Example – COUNT

- ▶ Command:
- ▶ `SELECT COUNT(*) FROM student;`
- ▶ Output (sample):
 count

 5
 (1 row)
- ▶ Meaning: there are 5 students in the table.



Databases and its Applications

SUM() and AVG()

- ▶ SUM(column) adds all non-NULL values.
- ▶ AVG(column) finds the mean of non-NULL values.
- ▶ Both are numeric aggregates.



Databases and its Applications

Example – SUM and AVG

- ▶ Suppose we have a table student(sid, sname, deptid, cgpa).
- ▶ Command:
- ▶ `SELECT SUM(cgpa), AVG(cgpa) FROM student;`
- ▶ Output (sample):

sum		avg
34.80		8.700

(1 row)



Databases and its Applications

MIN() and MAX()

- ▶ `MIN(column)` → smallest non-NULL value.
- ▶ `MAX(column)` → largest non-NULL value.
- ▶ Often combined with `WHERE` to find top students or lowest marks.



Example – MIN / MAX

- ▶ Command:
- ▶ `SELECT MIN(cgpa), MAX(cgpa) FROM student;`
- ▶ Output (sample):

min		max
7.80		9.50

(1 row)



Databases and its Applications

Combining WHERE with Aggregates

- ▶ You can aggregate only on filtered rows.
- ▶ Example:
- ▶ `SELECT COUNT(*) FROM student
WHERE deptid = 1;`
- ▶ Counts students only from department 1.



Databases and its Applications

GROUP BY – Why We Need It

- ▶ Aggregates normally return one row for the whole table.
- ▶ GROUP BY lets us get one result **per group**.
- ▶ Typical question: “How many students in each department?”



Databases and its Applications

GROUP BY – Syntax

- ▶ `SELECT column_to_show, aggregate_func`
`FROM table`
`GROUP BY column_to_show;`
- ▶ Every non-aggregated column in `SELECT` must appear in `GROUP BY`.



Example – GROUP BY deptid

- ▶ Command:
- ▶

```
SELECT deptid, COUNT(*) AS total_students  
FROM student  
GROUP BY deptid  
ORDER BY deptid;
```
- ▶ Output (sample):

deptid	total_students
1	2
2	1
3	1

(3 rows)



Databases and its Applications

GROUP BY with Join (Optional Now)

- ▶ When department names are in another table:
- ▶

```
SELECT d.deptname, COUNT(s.sid)
FROM department d
LEFT JOIN student s ON d.deptid = s.deptid
GROUP BY d.deptname;
```
- ▶ This shows count along with clear department names.



Databases and its Applications

HAVING – Filtering Groups

- ▶ WHERE filters **rows** before grouping.
- ▶ HAVING filters **groups** after GROUP BY.
- ▶ Use HAVING when the condition uses an aggregate.



Example – HAVING

- ▶ Command:
- ▶

```
SELECT deptid, COUNT(*) AS total
FROM student
GROUP BY deptid
HAVING COUNT(*) >= 2;
```

- ▶ Output (sample):

deptid	total
1	2

(1 row)

- ▶ Meaning: show only departments that have at least 2 students.



Databases and its Applications

Order of Evaluation (Important)

- ▶ PostgreSQL processes a SELECT in this logical order:
 - FROM
 - WHERE
 - GROUP BY
 - HAVING
 - SELECT
 - ORDER BY
- ▶ This is why you cannot use an aggregate directly in WHERE.



Databases and its Applications

NULL and Aggregates

- ▶ Aggregates ignore NULL values (except COUNT(*)).
- ▶ If a column has many NULLs, AVG() will use only non-NULL rows.
- ▶ To count rows with missing deptid: `SELECT COUNT(*) FROM student WHERE deptid IS NULL;`



Databases and its Applications

Practical Checks in psql

- ▶ `\dt` – to see tables before running queries.
- ▶ `\x` – expanded display for wide group results.
- ▶ Run each `SELECT` first without `WHERE`, then add filters.



PES
UNIVERSITY

CELEBRATING 50 YEARS

Pooja T S
Assistant Professor
Department of Computer Applications
poojats@pes.edu
080-26721983 Extn: 233